

Video Store Management System

1. Complete Commented Program

```
package com.company.oops;

import java.util.Arrays;

/*
 * Video class represents a single video in the video store.
 *
 * The class demonstrates:
 * 1. Encapsulation
 * 2. Constructor
 * 3. Instance variables
 * 4. Getter methods
 * 5. Object state management
 */
class Video {

    /*
     * private variables demonstrate ENCAPSULATION.
     *
     * These variables cannot be accessed directly from outside
     * the Video class.
     */
    private String title;
    private boolean flag;
    private float averageUserRating;

    /*
     * Constructor
     *
     * The constructor is called whenever a new Video object
     * is created.
     */
    Video(String title) {

        // Store the title received from the user.
        this.title = title;

        /*
         * Initially, the video is available in the store.
         *
         * true -> Video is available
         * false -> Video is checked out
         */
        flag = true;

        // Initially, the video has no rating.
        averageUserRating = 0.0f;
    }
}
```

```

/*
 * Method: beingCheckedOut()
 *
 * Changes the availability status of the video.
 *
 * false means that the video is currently checked out.
 */
void beingCheckedOut() {

    this.flag = false;
}

/*
 * Method: beingReturned()
 *
 * Changes the availability status of the video.
 *
 * true means that the video is available in the store.
 */
void beingReturned() {

    this.flag = true;
}

/*
 * Method: receiveRating()
 *
 * Stores a rating given by a user.
 *
 * This method can be extended later to calculate
 * the average rating of the video.
 */
void receiveRating(int rating) {

    System.out.println(
        "Rating received for " + this.title + ": " + rating
    );
}

/*
 * Getter method for title.
 *
 * Since title is private, other classes cannot directly
 * access it. They use this getter method instead.
 */
String getTitle() {

    return this.title;
}

/*
 * Getter method for availability status.

```

```

        */
        boolean getFlag() {

            return this.flag;
        }
    }

}

/*
 * VideoStore class manages multiple Video objects.
 *
 * It demonstrates:
 * 1. Array of objects
 * 2. Object creation
 * 3. Searching
 * 4. Stream API
 * 5. Lambda Expressions
 */
class VideoStore {

    /*
     * Array of Video objects.
     *
     * Maximum capacity of the store = 10 videos.
     */
    Video[] videoArr = new Video[10];

    /*
     * index keeps track of the position at which
     * the next video will be inserted.
     *
     * Initially, no video exists in the array.
     */
    int index = -1;

    /*
     * Method: addVideo()
     *
     * Adds a new video to the inventory.
     */
    void addVideo(String title) {

        /*
         * Move to the next available array position.
         */
        index++;

        /*
         * Create a new Video object and store its reference
         * in the array.
         */
        videoArr[index] = new Video(title);

        System.out.println("Video added: " + title);
    }
}

```

```

/*
 * Method: checkOut()
 *
 * Searches for the video by title and changes its
 * availability status to checked out.
 */
void checkOut(String title) {

    /*
     * Arrays.stream(videoArr)
     *
     * Converts the array into a Stream so that
     * Stream operations can be performed.
     */
    Arrays.stream(videoArr)

        /*
         * Some positions of the array may contain null.
         *
         * Therefore, remove null elements before accessing
         * any Video methods.
         */
        .filter(video -> video != null)

        /*
         * Select only the video whose title matches
         * the supplied title.
         */
        .filter(video -> video.getTitle().equals(title))

        /*
         * Perform the checkout operation on the
         * matching video.
         */
        .forEach(video -> video.beingCheckedOut());
}

/*
 * Method: returnVideo()
 *
 * Searches for a video by title and marks it as available.
 */
void returnVideo(String title) {

    Arrays.stream(videoArr)

        // Remove empty array positions.
        .filter(video -> video != null)

        // Find the video having the requested title.
        .filter(video -> video.getTitle().equals(title))

        // Mark the video as returned.
        .forEach(video -> video.beingReturned());
}

```

```

    }

    /*
    * Method: receiveRating()
    *
    * Passes the rating to the appropriate Video object.
    */
    void receiveRating(String title, int rating) {

        Arrays.stream(videoArr)

            // Ignore null elements.
            .filter(video -> video != null)

            // Find the required video.
            .filter(video -> video.getTitle().equals(title))

            // Pass the rating to the Video object.
            .forEach(video -> video.receiveRating(rating));
    }

    /*
    * Method: listInventory()
    *
    * Displays all videos that are currently available.
    */
    void listInventory() {

        System.out.println("\nList Inventory:");

        /*
        * Convert the array into a Stream.
        */
        Arrays.stream(videoArr)

            /*
            * Ignore empty positions in the array.
            */
            .filter(video -> video != null)

            /*
            * Display only videos that are available.
            *
            * getFlag() returns:
            * true -> available
            * false -> checked out
            */
            .filter(video -> video.getFlag())

            /*
            * Print the title of every available video.
            */
            .forEach(video -> System.out.println(video.getTitle()));
    }
}

```

```

/*
 * Main class
 *
 * This class is used to execute the program.
 */
public class VideoStoreLauncher {

    public static void main(String[] args) {

        /*
         * Create a VideoStore object.
         */
        VideoStore vsObj1 = new VideoStore();

        /*
         * Add videos to the store.
         */
        vsObj1.addVideo("Sample1");
        vsObj1.addVideo("Sample2");
        vsObj1.addVideo("Sample3");
        vsObj1.addVideo("Sample4");

        /*
         * Display all available videos.
         */
        vsObj1.listInventory();

        /*
         * Check out Sample3.
         *
         * Sample3 will no longer appear in the
         * available inventory.
         */
        vsObj1.checkOut("Sample3");

        vsObj1.listInventory();

        /*
         * Check out Sample1.
         */
        vsObj1.checkOut("Sample1");

        vsObj1.listInventory();

        /*
         * Return Sample3.
         *
         * Sample3 becomes available again.
         */
        vsObj1.returnVideo("Sample3");
    }
}

```

```
        vsObj1.listInventory();

        /*
         * Give a rating to Sample2.
         */
        vsObj1.receiveRating("Sample2", 5);
    }
}
```

2. Understanding the OOP Concepts

A. Class and Object

The program contains three classes:

```
Video
VideoStore
VideoStoreLauncher
```

Video

Represents one video.

```
Video video = new Video("Sample1");
```

Here:

- `Video` → class
- `video` → reference variable
- `new Video("Sample1")` → object creation

VideoStore

Represents the video store and manages multiple videos.

VideoStoreLauncher

Contains the `main()` method and starts program execution.

3. Encapsulation

The `Video` class contains:

```
private String title;  
private boolean flag;  
private float averageUserRating;
```

Because these variables are `private`, they cannot be directly accessed from another class.

For example, this is not allowed:

```
video.title = "Sample1";
```

Instead, the class provides methods:

```
String getTitle() {  
    return this.title;  
}
```

This is **encapsulation**:

Data is kept private and accessed through controlled methods.

4. Constructor

The constructor is:

```
Video(String title) {  
  
    this.title = title;  
    flag = true;  
    averageUserRating = 0.0f;  
}
```

When we write:

```
new Video("Sample1");
```

the constructor is automatically called.

The initial state becomes:

```
title = Sample1  
flag = true  
averageUserRating = 0.0
```

5. Understanding `this`

Consider:

```
Video(String title) {  
    this.title = title;  
}
```

There are two variables named `title`:

```
this.title  
title
```

`this.title` means:

`title` belonging to the current object.

The second `title` represents the constructor parameter.

Therefore:

```
this.title = title;
```

means:

Store the parameter value into the current object's title variable.

6. Object State

The variable:

```
private boolean flag;
```

maintains the availability state of a video.

```
flag = true  
    ↓  
Video available  
  
flag = false  
    ↓  
Video checked out
```

When:

```
beingCheckedOut();
```

is called:

```
this.flag = false;
```

When:

```
beingReturned();
```

is called:

```
this.flag = true;
```

This demonstrates how an object's **state can change during program execution**.

7. Array of Objects

The VideoStore contains:

```
Video[] videoArr = new Video[10];
```

This creates an array capable of storing references to 10 `Video` objects.

For example:

Index		Object
-----		-----
0	→	Video("Sample1")
1	→	Video("Sample2")
2	→	Video("Sample3")
3	→	Video("Sample4")
4	→	null
5	→	null
...		

This is an important example of an **array of objects**.

8. Why Do We Use `index`?

Initially:

```
int index = -1;
```

When the first video is added:

```
index++;
```

becomes:

```
index = 0
```

and:

```
videoArr[0] = new Video("Sample1");
```

When the second video is added:

```
index = 1
```

and:

```
videoArr[1] = new Video("Sample2");
```

So `index` keeps track of the last occupied position.

9. Stream API

The important statement is:

```
Arrays.stream(videoArr)
```

It converts the array into a **Stream**.

Conceptually:

```
Array
  ↓
Arrays.stream()
  ↓
Stream
  ↓
filter()
  ↓
filter()
  ↓
forEach()
```

A Stream provides a convenient way to process collections or arrays of data.

10. Why Is `filter(video -> video != null)` Required?

The array has a fixed size of 10.

Suppose we add only four videos:

```
0 → Sample1
1 → Sample2
2 → Sample3
3 → Sample4
4 → null
5 → null
6 → null
...
```

Therefore, the Stream contains `null` values.

If we directly write:

```
.filter(video -> video.getTitle().equals(title))
```

Java may try to execute:

```
null.getTitle()
```

which causes a:

```
NullPointerException
```

Therefore, we first write:

```
.filter(video -> video != null)
```

This removes all `null` references.

11. Understanding the Lambda Expression

Consider:

```
.filter(video -> video.getTitle().equals(title))
```

This Lambda can be read as:

For every `video`, check whether its title is equal to the requested `title`.

The structure is:

```
video
  ↓
parameter
  ->
    ↓
  lambda operator
video.getTitle().equals(title)
  ↓
condition
```

For example:

```
checkout("Sample3");
```

causes the condition to effectively check:

```
Sample1 == Sample3 → false
Sample2 == Sample3 → false
Sample3 == Sample3 → true
Sample4 == Sample3 → false
```

Only `Sample3` passes the filter.

12. Understanding `forEach()`

After filtering:

```
.forEach(video -> video.beingCheckedOut());
```

the matching video is passed to the Lambda.

It means:

For every video that passed the filter, call `beingCheckedOut()`.

This can also be written using a method reference:

```
.forEach(Video::beingCheckedOut);
```

Therefore:

```
.forEach(video -> video.beingCheckedOut());
```

and:

```
.forEach(Video::beingCheckedOut);
```

represent the same operation.

13. `listInventory()` – Complete Stream Pipeline

The most important example for students is:

```
Arrays.stream(videoArr)
    .filter(video -> video != null)
    .filter(video -> video.getFlag())
    .forEach(video -> System.out.println(video.getTitle()));
```

This can be explained in three stages.

Stage 1 – Convert Array to Stream

```
Arrays.stream(videoArr)
```

Stage 2 – Remove Null Values

```
.filter(video -> video != null)
```

Stage 3 – Select Available Videos

```
.filter(video -> video.getFlag())
```

Stage 4 – Display Titles

```
.forEach(video -> System.out.println(video.getTitle()));
```

So the complete logic is:

```
Video Array
    ↓
Create Stream
    ↓
Remove null objects
    ↓
Select available videos
    ↓
Display video titles
```

14. Why `filter()` Can Be Used Multiple Times

Students may ask why we don't combine both conditions.

The existing code uses:

```
.filter(video -> video != null)
.filter(video -> video.getFlag())
```

This is perfectly valid.

It can also be written as:

```
.filter(video -> video != null && video.getFlag())
```

However, the two-filter version is often easier for beginners to understand because each filtering condition has a separate purpose.

15. Traditional `for` Loop vs Stream

The original logic can be implemented using a traditional loop:

```
for (int i = 0; i <= index; i++) {
    if (videoArr[i] != null) {
        if (videoArr[i].getTitle().equals(title)) {
            videoArr[i].beingCheckedOut();
        }
    }
}
```

Using Stream API:

```
Arrays.stream(videoArr)
    .filter(video -> video != null)
    .filter(video -> video.getTitle().equals(title))
    .forEach(video -> video.beingCheckedOut());
```

The Stream version expresses the operation more declaratively:

Find the videos matching the condition and perform the operation on them.

16. Complete Program Flow

When the following statements execute:

```
vsObj1.addVideo("Sample1");  
vsObj1.addVideo("Sample2");  
vsObj1.addVideo("Sample3");  
vsObj1.addVideo("Sample4");
```

the store contains:

```
Sample1 → Available  
Sample2 → Available  
Sample3 → Available  
Sample4 → Available
```

After:

```
vsObj1.checkOut("Sample3");
```

the state becomes:

```
Sample1 → Available  
Sample2 → Available  
Sample3 → Checked Out  
Sample4 → Available
```

Therefore:

```
vsObj1.listInventory();
```

displays:

```
Sample1  
Sample2  
Sample4
```

Then:

```
vsObj1.checkOut("Sample1");
```

changes the state to:

```
Sample1 → Checked Out  
Sample2 → Available  
Sample3 → Checked Out  
Sample4 → Available
```

Therefore the inventory becomes:

```
Sample2  
Sample4
```

Finally:

```
vsObj1.returnVideo("Sample3");
```

changes:

```
Sample3 → Available
```

and the inventory becomes:

```
Sample2  
Sample3  
Sample4
```

17. Important Concepts Covered

This single program can be used to explain several Java concepts:

Concept	Where Used
Class	Video, VideoStore
Object	new Video("Sample1")
Constructor	Video(String title)
Encapsulation	private variables
this keyword	this.title, this.flag
Getter	getTitle(), getFlag()
Array of Objects	Video[] videoArr
Object State	flag
Stream API	Arrays.stream()
Lambda Expression	video -> ...
filter()	Selecting required videos
forEach()	Performing an operation
Method Reference	Video::beingCheckedOut
Null Handling	video != null

18. Recommended Teaching Sequence

For students who are learning Lambda and Stream API, explain this program in the following order:

1. Class and Object
- ↓
2. Constructor
- ↓
3. Encapsulation
- ↓
4. Array of Objects
- ↓
5. Object State
- ↓
6. Traditional for-loop implementation
- ↓
7. Stream API
- ↓
8. `filter()`
- ↓
9. Lambda Expression
- ↓
10. `forEach()`
- ↓
11. Method Reference

This sequence helps students understand **why** Streams and Lambdas are being used instead of simply learning the syntax.

Key Classroom Explanation

A concise explanation that can be given to students is:

The `video` class represents an individual video, while the `VideoStore` class manages an array of `Video` objects. The Stream API is used to process the array, `filter()` is used to select the required videos, Lambda Expressions define the filtering and processing logic, and `forEach()` performs the required operation on each matching video.